Platform Navigation — Defining Behavior Before Visuals (1)

Platform Navigation Redesign

Role: UI Designer — Interaction architecture and design system alignment

Year: 2025

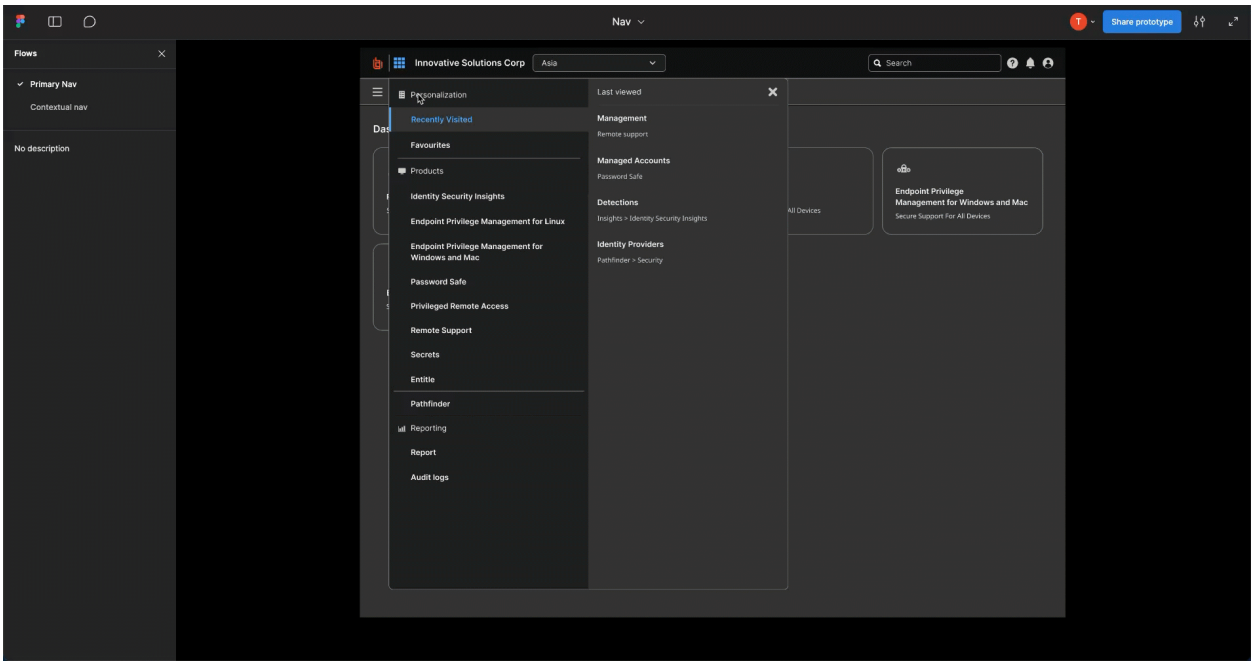
Overview

I joined the team to address a critical scalability challenge: our platform navigation was inconsistent and difficult to expand. Across different products, the lack of shared standards created technical debt for developers and a fragmented experience for users.

My focus was on three key goals:

- **Standardize behavior:** Define clear rules for navigation, specifically for pinning and unpinning.
- **Align with the system:** Ensure full compliance with Shield Design System (SDS) tokens.
- **Bridge the gap to code:** Transform conceptual designs into a production-ready architecture.

The challenge was primarily architectural, not just visual. The system struggled to manage hierarchy and complex tenant switching. I led the collaboration with engineering to define the necessary logic, ensuring the new navigation was both scalable and technically feasible.



Team exploration of AWS-inspired navigation ideas

Problem

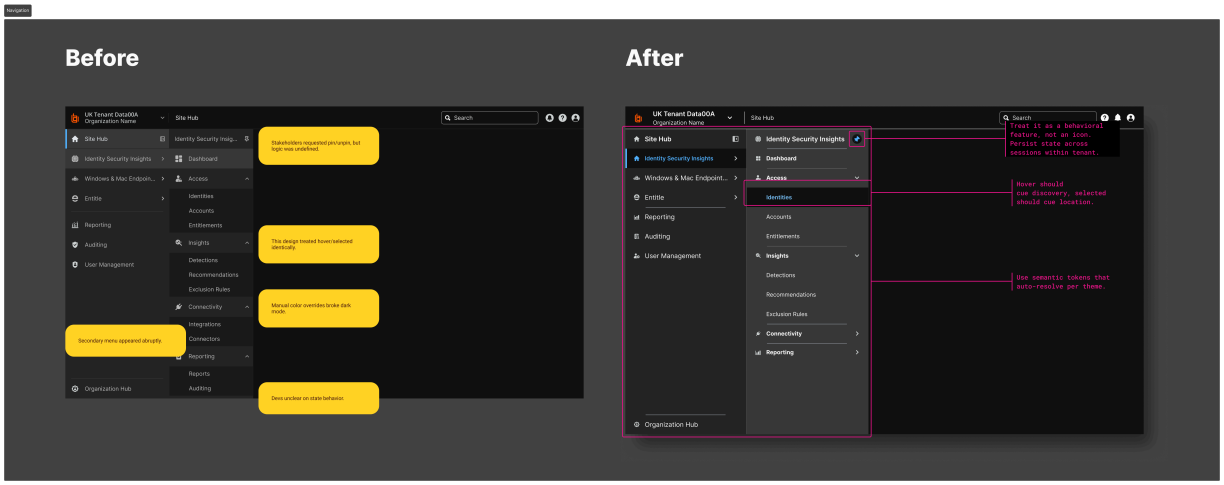
As our platform expanded to multiple products, the navigation system became a bottleneck rather than an enabler. Users faced a disjointed experience where behavior changed from one product to another, leading to confusion and lower task efficiency.

For the business, this fragmentation was costly. Developers struggled to maintain disparate codebases, and new product integrations were slowed down by the need to "reinvent" navigation each time. The lack of defined interaction logic meant that what looked like a simple UI update often turned into a complex engineering headache.

Key Issues:

- **Fragmented User Experience:** Inconsistent behavior across products increased cognitive load.
- **Technical Debt:** No shared logic for states like pin/unpin or persistence.

- **Operational Inefficiency:** Engineering spent excessive time fixing "styling" issues that were actually undefined behaviors.



How the design from UX Designer had some key issues.

The Solution: A Token-Driven, Scalable Architecture

I treated navigation as a product with its own logic, creating a single model to serve the entire ecosystem.

1. Interaction Logic Over Visuals

I replaced static mocks with interactive prototypes to document the full lifecycle—collapsing, expanding, and pinning. This aligned stakeholders on *behavior*, ensuring the system felt predictable.

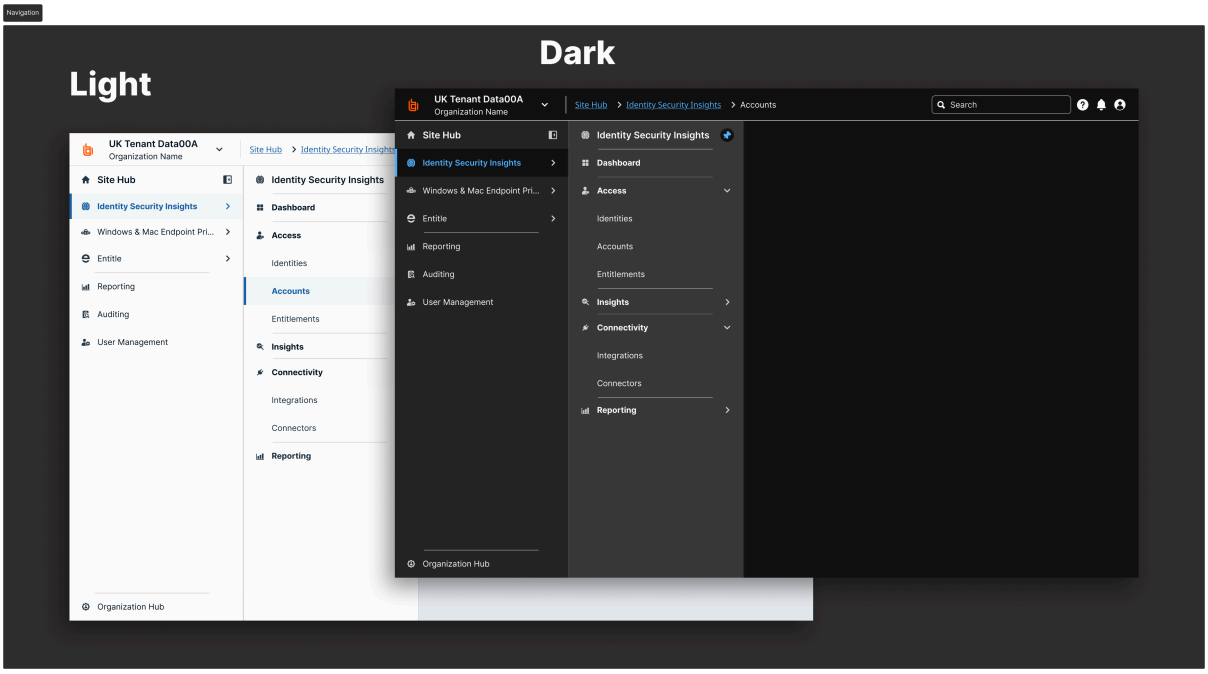
2. Standardized Pin/Unpin

I designed a model balancing density with clarity:

- **Clear Actions:** Explicit "Pin/Unpin" labels removed ambiguity.
- **System Alignment:** Used SDS tokens (`icon.action.active`) for built-in accessibility and theming.

3. Bridging Design & Code

I delivered a "State Map" detailing every state (Hover, Focus, Pinned), ensuring a seamless handoff that allowed the UXA team to build a functional POC immediately.



Light and dark mode design for the navigation.

My Approach

I focused on turning a static concept into a **scalable, token-driven interaction system** that could be implemented and maintained across products.

1. **Mapped real usage patterns**

   Partnered with engineers and designers to understand how people moved between tenants and used pinned items.

2. **Defined clear interaction logic**

   Documented what should happen when users collapse, expand, pin, unpin, and switch tenants. I used Figma variants and prototypes so the behavior was easy to see and test.

3. **Integrated SDS token architecture**

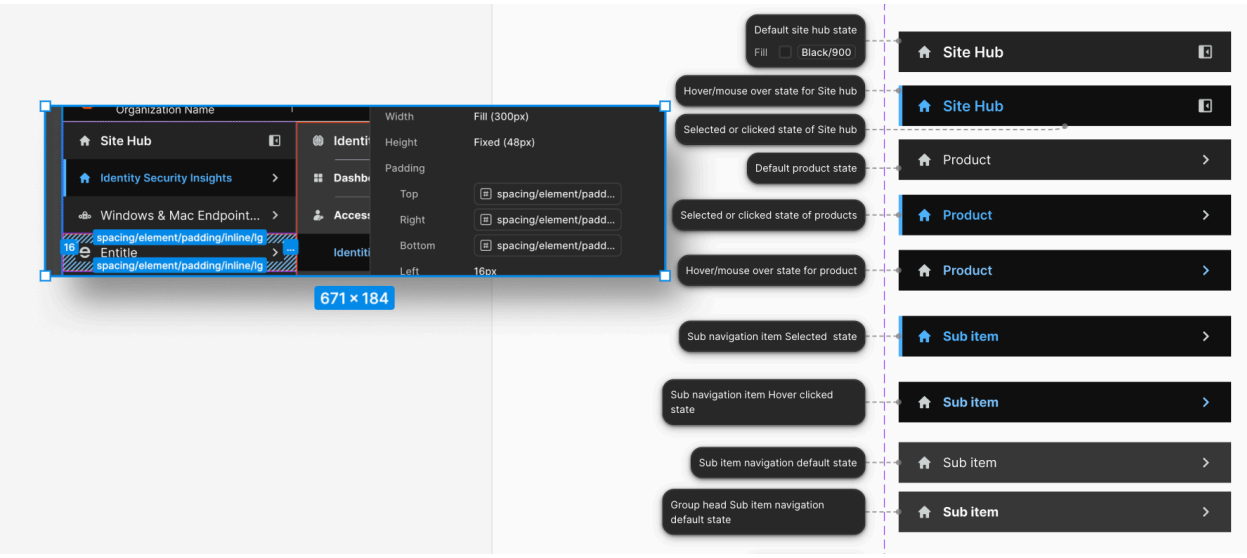
   Mapped colors, typography, spacing, and elevation to semantic tokens. This kept the nav consistent in light and dark mode and aligned it with the design system.

4. **Refined information hierarchy**

   Adjusted type size, contrast, and spacing to improve readability and rhythm.

5. **Validated with engineering**

   Partnered with the UXA team to test persistence, animation, and cross-tenant behavior in a proof of concept.

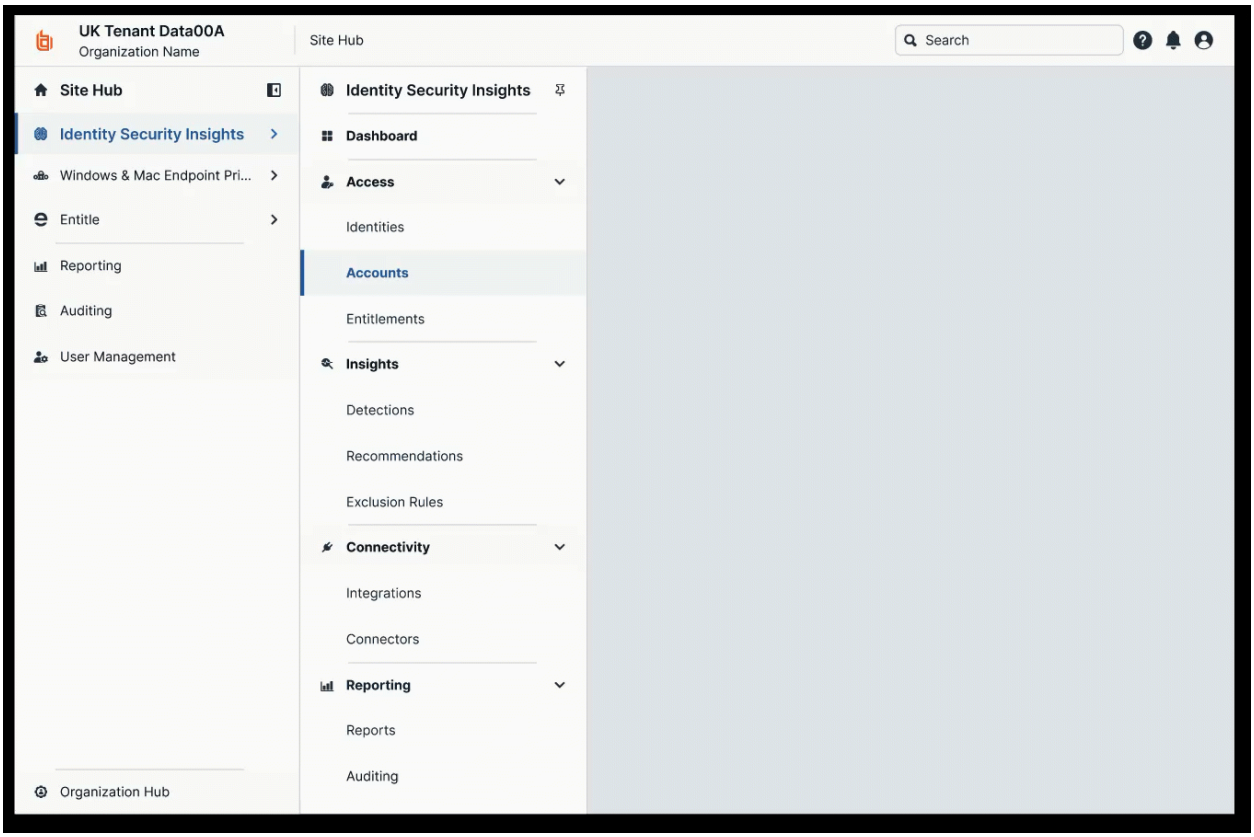


Interaction logic for the navigation states

Designing and documentation

Prototyping creative solutions for the pinned state

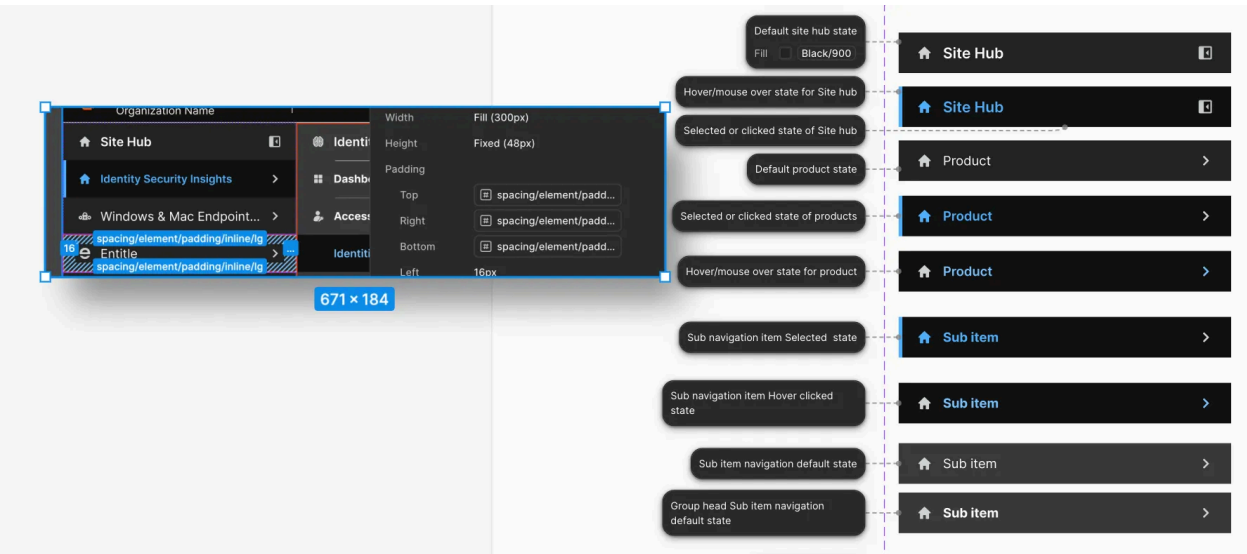
To tackle the challenge of visually representing the pinned state, I crafted three distinct prototypes: a **solid pin** for clarity, an **outline pin with an accent ring** for dark mode visibility, and a **dynamic indicator** using motion for feedback. I ensured every version was fully interactive in Figma and linked to design tokens, allowing engineering to begin testing immediately. Once the preferred option was selected, I finalized the model to balance usability and feasibility. This design features **action-based labels**, **instant persistent feedback**, and **SDS-aligned color tokens** with accessible 160ms fade and scale transitions.



Pin/Unpin interaction is the product

Interaction States and Handoff

I documented every navigation state—Hover, Focus, Active, Pinned, Expanded, and Collapsed—to ensure consistent behavior across products. This comprehensive interaction model, covering both light and dark mode designs, provided the UXA team with a clear state map to plug directly into their proof of concept.

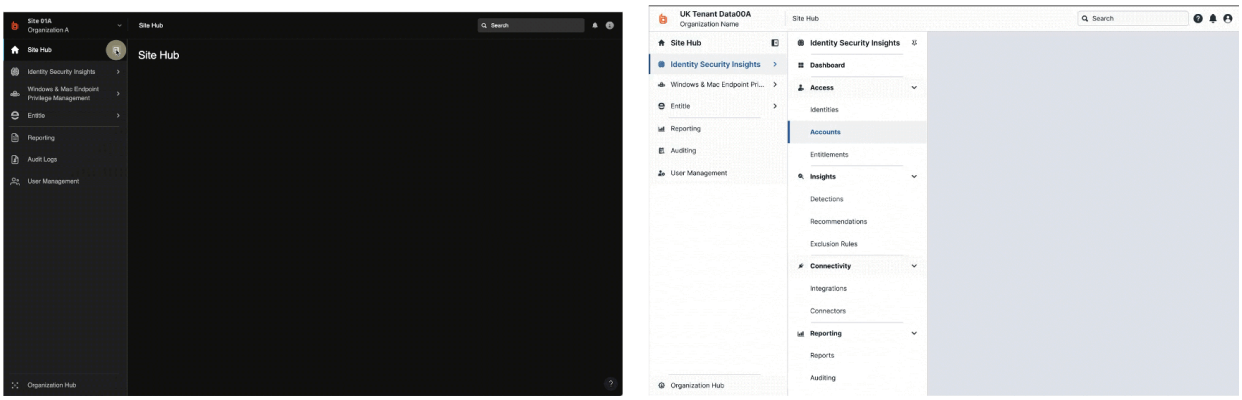


Interaction logic for the navigation states

The Impact: Accelerated Delivery & User Trust

The redesign did more than clean up the interface; it operationalized navigation as a reusable system component.

- **Business Impact:** Accelerated new product integration by providing a "drop-in" navigation module, significantly reducing frontend dev time.
- **User Success:** Established a consistent mental model for users moving between products, reducing navigation errors and friction.
- **Operational Efficiency:** Eliminated the "logic gaps" in handoff, cutting down QA cycles related to navigation bugs.
- **System adoption:** The logic defined here became the standard for future platform components, proving the value of a token-driven interaction model.

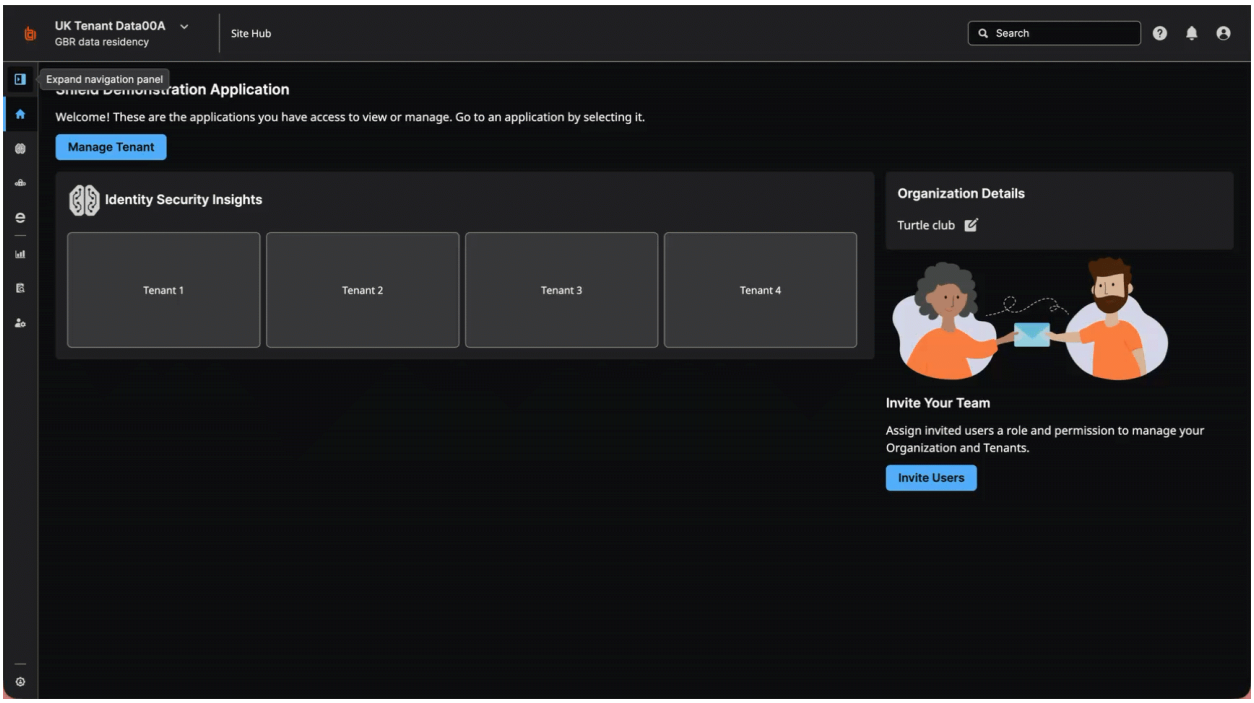


Comparison of before and after design of the navigation.

Building clarity through navigation: A reflection

I learned that polished visuals fail if behavior is unclear. My goal was to create a system with straightforward rules so teams could build confidently and users could rely on consistency. I transformed a static prototype into an implementation-ready framework that gave the team clear direction.

A key refinement was the site-switching experience. Initially, complex controls confused users about their location. By simplifying the tenant switcher from a search box to a dropdown and separating navigation actions, I aligned the interface with actual usage patterns—proving that clarity often comes from subtraction.



Reflection

Visual design fails if behavior is confusing. My goal was to define clear rules for interactions like pinning and unpinning so teams knew how to build them. I turned a static prototype into a trusted, implementation-ready framework.

I also refined site switching by aligning controls with real usage. We replaced a complex 'search-style' switcher with a simple dropdown and separated navigation actions. This removed ambiguity about platform location and behavior, creating a UI that matched how tenants actually work.

Learnings:

- System problems require cross-functional collaboration and deep technical understanding.
- Designing for scalability means anticipating future use cases and team needs.
- Clear, codified rules remove ambiguity and foster confidence across teams.

Next Steps:

- User testing to validate the new navigation across varied personas.
- Ongoing documentation and training for product teams and developers.
- Iterative improvements based on analytics and direct feedback post-launch.

Business Impact

The new navigation system delivered immediate value by reducing inconsistencies, empowering development teams, and creating a more reliable user experience. Product teams can now launch features faster with less risk of regression, and users benefit from a predictable and coherent journey across products. This solution sets the stage for future growth, enabling the platform to scale seamlessly alongside new offerings.